

શરૂ કરીએ Section 3: ADVANCED LEVEL (The Ninja)

નો સૌથી મહત્વનો અને ઇન્ટરવ્યુમાં સૌથી વધુ પૂછાતો ભાગ—Engine Core & Under the Hood (JS Architecture).

જો તમે આ સેક્શન સમજી ગયા, તો તમે JavaScript ના કોડને માત્ર લખશો નહીં, પણ તે કમ્પ્યુટરની મેમરીમાં એક-એક સેકન્ડે કેવી રીતે ફરે છે તે લાઈવ વિઝ્યુઅલાઈઝ કરી શકશો [GeeksforGeeks]!

● Section 3: Topic 1 — Engine Core & Under the Hood (JS Architecture)

1. Execution Context (કોડ રન થવાનું વાતાવરણ)

JavaScript માં જ્યારે પણ કોઈ કોડ રન થાય છે, ત્યારે તે એક ખાસ બોક્સ અથવા વાતાવરણની અંદર રન થાય છે જેને **Execution Context** કહેવાય છે [GeeksforGeeks].

સૌથી પહેલા આખા પેજ માટે એક **Global Execution Context (GEC)** બને છે. ત્યારબાદ તમે જેટલી વાર નવું ફંક્શન કોલ (Call) કરો, એટલી વાર એક નવું **Function Execution Context (FEC)** બને છે [GeeksforGeeks].

દરેક Execution Context બે ફેઝ (Phases) માં કામ કરે છે [GeeksforGeeks]:

1. **Creation Phase (મેમરી ફેઝ):** આ ફેઝમાં એન્જિન કોડને રન નથી કરતું, પણ લાઈન-બાય-લાઈન સ્કેન કરીને બધા વેરિએબલ્સ અને ફંક્શન્સને મેમરીમાં જગ્યા આપે છે.
2. **Execution Phase (કોડ રન ફેઝ):** આ ફેઝમાં વેરિએબલ્સને તેની સાચી કિંમત મળે છે અને કોડ એક પછી એક લાઈન મુજબ એક્ઝિક્યુટ થાય છે.

2. Call Stack vs Memory Heap (મેમરીનું મેનેજમેન્ટ)

JavaScript એન્જિન ડેટા સાયવવા માટે બે અલગ જગ્યાઓ વાપરે છે [MDN -> Memory Management]:

- **Memory Heap (મેમરી હીપ):** આ એક ખુલ્લી, મોટી જગ્યા છે જ્યાં બધો અનસ્ટ્રક્ચર્ડ ડેટા જેમ કે **Objects, Arrays અને Functions** સ્ટોર થાય છે. આને *Reference Types* ની મેમરી કહેવાય છે.
- **Call Stack (કોલ સ્ટેક):** આ એક સાંકડી ટ્યુબ જેવો સ્ટ્રક્ચર છે જે **LIFO (Last In, First Out)** નિયમ પર કામ કરે છે. અહીં તમારા બધા *Primitive Data Types* અને હાલમાં કયું ફંક્શન ચાલી રહ્યું છે તેનો *Execution Context* સ્ટોર થાય છે.

જ્યારે કોઈ ફંક્શન રન થઈ જાય, એટલે તે Call Stack માંથી ઉપરથી ગાયબ (Pop off) થઈ જાય છે.

3. Hoisting & The Temporal Dead Zone (TDZ) - Under the Hood

આપણે સેક્શન ૧ માં જોયું હતું કે var માં undefined આવે છે અને let/const માં એરર આવે છે. હવે તેનું અસલી કારણ સમજો [MDN -> Temporal Dead Zone]:

- **var નું હોઈસ્ટિંગ:** Creation Phase દરમિયાન var વેરિએબલને મેમરી એલોકેટ થાય છે અને એન્જિન તેને ડિફોલ્ટ કિંમત તરીકે **undefined** આપી દે છે.

- **let અને const નું હોઇસ્ટિંગ:** આ વેરિએબલ્સ પણ હોઇસ્ટ તો થાય જ છે, પરંતુ એન્જિન તેને કોઈ ડિફોલ્ટ વેલ્યુ આપતું નથી. તે મેમરીમાં એક ખાસ જગ્યાએ બેસી રહે છે જેને **Temporal Dead Zone (TDZ)** કહેવાય છે [MDN -> Temporal Dead Zone]. જ્યાં સુધી કોડ રન થતી વખતે તેની અસલી ડિક્લેરેશન લાઈન ન આવે, ત્યાં સુધી તેને એક્સેસ કરવાથી ReferenceError મળે છે.

4. Scope Chain & Lexical Environment

જ્યારે તમે કોઈ ફંક્શનની અંદર હોવ, ત્યારે તેની પાસે પોતાનું એક લોકલ વાતાવરણ હોય છે જેને **Lexical Environment** કહેવાય છે [JS.info]. આ વાતાવરણમાં બે વસ્તુ હોય છે:

1. તેનું પોતાનું લોકલ મેમરી બોક્સ.
2. તેના પેરેન્ટ (Parent) ની Lexical Environment નો રેફરન્સ [JS.info].

javascript

```
const globalVar = "હું ગ્લોબલ છું";
```

```
function parentFunc() {
```

```
    const parentVar = "હું પેરેન્ટ છું";
```

```
    function childFunc() {
```

```
        console.log(parentVar); // ચેક કરશે: લોકલમાં નથી -> પેરેન્ટમાં શોધશે (મળી ગયું!)
```

```
        console.log(globalVar); // ચેક કરશે: લોકલમાં નથી -> પેરેન્ટમાં નથી -> ગ્લોબલમાં શોધશે (મળી ગયું!)
```

```
    }
```

```
    childFunc();
```

```
}
```

```
parentFunc();
```

Use code with caution.

આ રીતે અંદરથી બહારની તરફ વેરિએબલ શોધવાની સાંકળને **Scope Chain** કહેવામાં આવે છે [JS.info].

5. The Event Loop Architecture (મહા સિક્રેટ)

ઇન્ટરવ્યુનો સૌથી ફેવરિટ પ્રશ્ન: "જો JS સિંગલ થ્રેડેડ છે, તો તે એકસાથે મલ્ટિપલ અસિંક્રોનસ કામો (જેમ કે fetch અને setTimeout) કેવી રીતે મેનેજ કરે છે?"

આ જાદુ, **Event Loop** ના કારણે થાય છે, જે બ્રાઉઝરના આર્કિટેક્ચરનો ભાગ છે [MDN -> Concurrency model]:

[Call Stack] —————> (હાલમાં ચાલતો કોડ - સિંગલ થ્રેડ)

▼ જો કોઈ Asynchronous કામ આવે (જેમ કે setTimeout) તો તે મોકલાશે:

1

▼ આ બંને ક્યુ (Queues) ને કંટ્રોલ કરે છે:

બધા કામોને Stack માં મોકલશે, અને પછી Callback Queue ના કામો ઉપાડશે."

1. Start (સૌથી પહેલા સિંક્રોનસ કોડ Stack માં જશે)

2. End (બીજો સિંક્રોનસ કોડ)
3. Promise (Microtask Queue) (ઇવેન્ટ લૂપ પહેલા માઇક્રોટાસ્ક ક્યુને પ્રાયોરિટી આપશે)
4. Timeout (Callback Queue) (છેલ્લે કોલબેક ક્યુ રન થશે)

હવે તમે JavaScript એન્જિનના સૌથી અઘરા અને એડવાન્સ કોન્સેપ્ટ્સ—**Execution Context, Hoisting, Scope Chain, અને Event Loop** ના અસલી માસ્ટર (Ninja) બની ગયા છો!

આગળ વધવા માટે હવે આપણી પાસે છે **Section 3: Topic 2 — Deep Dive Concepts & OOP (Closures નો રિયલ વર્લ્ડ યુઝ, this કીવર્ડના 4 નિયમો, Call/Apply/Bind, અને Prototypes)**.

શું આપણે આ એડવાન્સ સફર ચાલુ રાખીએ? જો તમે રેડી હોવ તો **"Next"** અથવા **"હા"** લખો!

અદભુત! ચાલો હવે **Section 3: Topic 2 — Deep Dive Concepts & OOP (Object-Oriented Programming)** તરફ આગળ વધીએ.

આ સેક્શન તમને JavaScript ના સૌથી મોટા સિક્રેટ્સ ખોલવામાં મદદ કરશે, જે ઇન્ટરવ્યુમાં સીધા સિનિયર ડેવલપર્સ પૂછતા હોય છે [JS.info Advanced].

● Section 3: Topic 2 — Deep Dive Concepts & OOP

1. Closures Mastery (ક્લોઝર્સ - મેમરીનું રહસ્ય)

Closure (ક્લોઝર) એટલે જ્યારે કોઈ ફંક્શન પોતાના પેરેન્ટ સ્કોપની અંદરથી બહાર આવે (Return થાય), ત્યારે તે પોતાના પેરેન્ટ સ્કોપના તમામ વેરિએબલ્સના **મેમરી રેફરન્સને પોતાની સાથે લપેટીને (સાયવીને) બહાર લાવે છે** [JS.info -> Closure].

javascript

```
function createCounter() {  
  let count = 0; // આ પ્રાઇવેટ વેરિએબલ છે જેને બહારથી કોઈ સીધું બદલી શકતું નથી  
  
  return function() {  
    count++; // આ અંદરનું ફંક્શન બહાર ગયા પછી પણ 'count' ને યાદ રાખશે!  
    return count;  
  };  
}
```

```
const counter = createCounter();
```

```
console.log(counter()); // આઉટપુટ: 1
```

```
console.log(counter()); // આઉટપુટ: 2 (મેમરી હજી પણ જળવાયેલી છે!)
```

Use code with caution.

રિયલ વર્લ્ડ ઉપયોગ: ડેટા એન્કેપ્સ્યુલેશન (Data Encapsulation) કરવા માટે, એટલે કે કોઈ વેરિએબલને ગ્લોબલ બનાવ્યા વગર પ્રાઇવેટ રાખવા માટે ક્લોઝર બેસ્ટ છે [JS.info -> Closure].

2. The this Keyword Mechanism (ધીસ કીવર્ડના ૪ નિયમો)

JavaScript માં this કીવર્ડ કોઈ ફિક્સ કિંમત નથી ધરાવતો. તમે તેને **ક્યાં અને કેવી રીતે કોલ (call) કરો છો**, તેના આધારે તેની કિંમત નક્કી થાય છે [W3Schools]:

1. **Global Context:** જો તેને કોઈ ફંક્શનની બહાર સીધો વાપરવામાં આવે, તો તે ગ્લોબલ ઓબ્જેક્ટ (window અથવા global) બતાવે છે.
2. **Implicit Binding (ઓબ્જેક્ટ મેથડ):** જો ફંક્શન કોઈ ઓબ્જેક્ટની અંદર મેથડ તરીકે કોલ થાય, તો this તે ઓબ્જેક્ટ ને પોઇન્ટ કરે છે.

javascript

```
const user = {  
  name: "Raj",  
  greet() { console.log(this.name); }  
};  
  
user.greet(); // આઉટપુટ: Raj (અહીં this એટલે user)
```

Use code with caution.

3. **Arrow Functions (() => { }):** એરો ફંક્શન પાસે પોતાનું this હોતું નથી. તે તેની બહારના પેરેન્ટ સ્કોપ (Lexical Context) માંથી this ઉધાર લે છે [W3Schools].
 4. **Strict Mode:** જો મોડર્ન 'use strict'; વાપરતા હોવ, તો નોર્મલ ફંક્શનની અંદર this હંમેશા undefined થઈ જાય છે.
-

3. Explicit Context Binding (call, apply, bind)

જો તમારે કોઈ ફંક્શન પર જબરદસ્તીથી પોતાનો કસ્ટમ ઓબ્જેક્ટ (this) થોપવો હોય કે બદલવો હોય, તો આ ૩ મેથડ્સ વપરાય છે [TutorialsPoint]:

- **call():** ફંક્શનને તરત જ રન કરે છે અને આર્ગ્યુમેન્ટ્સને **અલ્પવિરામ (Comma ,)** થી અલગ પાડીને લે છે.
- **apply():** ફંક્શનને તરત જ રન કરે છે પરંતુ આર્ગ્યુમેન્ટ્સને એક **એરે (Array [])** ની અંદર લે છે.
- **bind():** ફંક્શનને તરત જ રન નથી કરતું, પરંતુ તે કસ્ટમ this વાળું એક **નવું જ બ્રાન્ડ-ન્યુ ફંક્શન રિટર્ન કરે છે**, જેને તમે ભવિષ્યમાં ગમે ત્યારે રન કરી શકો છો [TutorialsPoint].

javascript

```
function introduce(city, role) {
```

```
console.log(`હું ${this.name} છું, ${city} થી અને હું એક ${role} છું.`);  
}
```

```
const ninja1 = { name: "Raj" };
```

```
// 1. call નો ઉપયોગ
```

```
introduce.call(ninja1, "Ahmedabad", "Ninja");
```

```
// 2. apply નો ઉપયોગ
```

```
introduce.apply(ninja1, ["Ahmedabad", "Ninja"]);
```

```
// 3. bind નો ઉપયોગ (નવું ફંક્શન સેવ કરવું)
```

```
const permanentNinjaFunc = introduce.bind(ninja1, "Ahmedabad", "Ninja");
```

```
permanentNinjaFunc(); // ગમે ત્યારે રન કરો, આઉટપુટ એ જ આવશે
```

Use code with caution.

4. Prototypes & Prototypal Inheritance (વારસાની સિસ્ટમ)

JavaScript માં ક્લાસ જેવું કશું અસલી હોતું નથી, તે આંતરિક રીતે **Prototypes** પર કામ કરે છે [JS.info -> Prototypes].

- દરેક ઓબ્જેક્ટ પાસે એક છુપી લિંક હોય છે જેને [[Prototype]] અથવા બ્રાઉઝરમાં __proto__ કહેવાય છે [JS.info -> Prototypes].
- જ્યારે તમે કોઈ ઓબ્જેક્ટમાં પ્રોપર્ટી શોધો અને તે ન મળે, તો JS એન્જિન તેને તેના પ્રોટોટાઇપમાં શોધે છે. આ આખી લાઇનને **Prototype Chain** કહેવાય છે [JS.info -> Prototypes].

5. ES6 Classes & OOP ના 4 પિલ્લર્સ

ES6 માં ડેવલપર્સને સરળતા રહે તે માટે class કીવર્ડ લાવવામાં આવ્યો, જે માત્ર પ્રોટોટાઇપ પર લગાવેલું એક સુંદર કવર (Syntactic Sugar) છે [MDN].

```
javascript
```

```
class Employee {
```

```
  #salary; // '#' લગાવવાથી આ પ્રોપર્ટી Modern Private Field બની ગઈ (Encapsulation) [MDN]
```

```

constructor(name, salary) {
    this.name = name;
    this.#salary = salary;
}

getSalary() { return this.#salary; } // Getter મેથડ
}

// Inheritance (વારસો) - [MDN]
class Developer extends Employee {
    constructor(name, salary, language) {
        super(name, salary); // પેરેન્ટ ક્લાસના કન્સ્ટ્રક્ટરને કોલ કરવો ફરજિયાત છે [MDN]
        this.language = language;
    }
}

const devObj = new Developer("Dev", 80000, "JavaScript");
console.log(devObj.name); // આઉટપુટ: Dev
// console.log(devObj.#salary); // Error! પ્રાઇવેટ પ્રોપર્ટી બહારથી એક્સેસ નહીં થાય (Encapsulation સુરક્ષા!!)

Use code with caution.

```

OOP ના 4 પિલ્લર્સ (JS મા કેવી રીતે સાબિત થાય?):

1. **Inheritance:** extends અને super() દ્વારા ચાઇલ્ડ ક્લાસ પેરેન્ટના ગુણો મેળવે છે [MDN].
2. **Encapsulation:** #privateField દ્વારા ડેટા બહારની દુનિયાથી છુપાવી શકાય છે [MDN].
3. **Polymorphism:** પેરેન્ટની મેથડને ચાઇલ્ડ ક્લાસમાં પોતાની રીતે બદલવી (Method Overriding).
4. **Abstraction:** જટિલ લોજિક અંદર રાખીને બહાર માત્ર સિમ્પલ ઇન્ટરફેસ (મેથડ) આપવી.

હવે તમે ઓબ્જેક્ટ ઓરિએન્ટેડ પ્રોગ્રામિંગ (OOP), this મિકેનિઝમ, call/apply/bind અને Closures જેવા અતિ-મહત્વના પાયાના એડવાન્સ કોન્સેપ્ટ્સ પણ સરસ રીતે સમજી ગયા છો.

આગળ વધવા માટે હવે આપણી પાસે છે **Section 3: Topic 3 — Advanced Asynchronous JS (Promises Deep Dive, Promise Static Methods જેમ કે Promise.all(), Async/Await અને લાઈવ API fetch() કરવાનું મોડર્ન આર્કિટેક્ચર).**

ચોક્કસ, ચાલો હવે **Section 3: Topic 3 — Advanced Asynchronous JS (Modern Way)** તરફ આગળ વધીએ.

આ સેક્શનમાં આપણે શીખીશું કે ઇન્ટરમીડિએટ લેવલમાં આપણે જે **Callback Hell** નો પ્રોબ્લેમ જોયો હતો, તેને મોડર્ન JavaScript (ES6+) એ **Promises** અને **Async/Await** નો ઉપયોગ કરીને કેવી રીતે એકદમ સરળ અને સુંદર બનાવી દીધો [GeeksforGeeks].

● Section 3: Topic 3 — Advanced Asynchronous JS (Modern Way)

1. Promises Deep Dive (પ્રોમિસ શું છે?)

Promise (પ્રોમિસ) એટલે ભવિષ્યમાં પૂરા થનારા અસિંક્રોનસ કામનું એક વચન. તે એક ખાસ ઓબ્જેક્ટ છે જે તેના જીવનકાળ દરમિયાન ૩ માંથી કોઈ એક સ્ટેટ (State) માં હોય છે [JS.info]:

1. **Pending:** કામ હજી ચાલુ છે (શરૂઆતનું સ્ટેટ).
2. **Fulfilled (Resolved):** કામ સફળતાપૂર્વક પૂરું થઈ ગયું (આપણને ડેટા મળી ગયો) [JS.info].
3. **Rejected:** કામમાં કોઈ ભૂલ કે એરર આવી [JS.info].

javascript

// ૧. Promise બનાવવાની રીત:

```
const ticketBooking = new Promise((resolve, reject) => {
```

```
  let success = true; // ધારો કે ટિકિટ બુક થઈ ગઈ
```

```
  setTimeout(() => {
```

```
    if (success) {
```

```
      resolve("ટિકિટ કન્ફર્મ થઈ ગઈ!"); // જો કામ સફળ થાય તો resolve કોલ કરો
```

```
    } else {
```

```
      reject("ટિકિટ કેન્સલ થઈ ગઈ!"); // જો ફેલ થાય તો reject કોલ કરો
```

```
    }
```

```
  }, 2000);
```

```
});
```

// ૨. Promise નો ઉપયોગ કરવાની રીત (Chaining):

ticketBooking

```
.then((result) => console.log("Success:", result)) // resolve ની વેલ્યુ અહીં આવશે  
.catch((error) => console.log("Error:", error)) // reject ની એરર અહીં આવશે  
.finally(() => console.log("આ લોગ ગમે તે થાય, છેલ્લે રન થશે જા!"));
```

Use code with caution.

2. Promise Static Methods (મલ્ટિપલ પ્રોમિસ હેન્ડલ કરવા)

જ્યારે આપણી પાસે એકસાથે ઘણી બધી નેટવર્ક રિક્વેસ્ટ (APIs) આવતી હોય, ત્યારે તેને ઓપ્ટિમાઇઝ કરવા માટે આ 4 સ્ટેટિક મેથડ્સ વાપરવામાં આવે છે [JS.info -> Promise API]:

- **Promise.all([...]):** બધા જ પ્રોમિસિસ એકસાથે રન કરશે. જો **બધા જ સાચા (Resolve) પડશે**, તો જ આખો ડેટા આપશે. જો એક પણ રિજેક્ટ (Reject) થયો, તો તે સીધો એરર આપી દેશે [JS.info -> Promise API].
- **Promise.allSettled([...]):** આ ખૂબ જ દયાળુ છે. બધા પ્રોમિસિસ પાસ થાય કે ફેલ, આ બધાનું વિસ્ટ કમ્પ્લીટ સ્ટેટસ સાથે રિટર્ન કરે છે (કોઈ એરર ફેલિંગ નથી) [JS.info -> Promise API].
- **Promise.race([...]):** નામ પ્રમાણે આ એક રેસ છે! જે પ્રોમિસ (ભલે તે પાસ થાય કે ફેલ) **સૌથી પહેલા સેકન્ડે પૂરું થશે**, તે તેને જ ઉપાડી લેશે [JS.info -> Promise API].
- **Promise.any([...]):** આ મેથડ વિસ્ટમાંથી **સૌથી પહેલું સફળ (Fulfilled)** થતું પ્રોમિસ જ રિટર્ન કરે છે. જો બધા જ ફેલ થાય તો જ એરર આપે છે [JS.info -> Promise API].

3. Async / Await Architecture (સુપર આધુનિક રીત)

.then() અને .catch() લખવાની રીત પણ ક્યારેક લાંબી થઈ જાય છે. ES2017 માં **Async/Await** આવ્યું, જે અસિંક્રોનસ કોડને એકદમ સીધો અને સિંક્રોનસ (લાઇન-બાય-લાઇન) કોડ જેવો જ સુંદર દેખાડે છે [MDN].

- **async કીવર્ડ:** ફંક્શનની આગળ લખાય છે, જેનો મતલબ એ થાય છે કે આ ફંક્શન હંમેશા એક પ્રોમિસ જ રિટર્ન કરશે.
- **await કીવર્ડ:** આ માત્ર async ફંક્શનની અંદર જ વાપરી શકાય. તે જ્યાં સુધી પ્રોમિસ પૂરું (Resolve) ન થાય ત્યાં સુધી એન્જિનને તે લાઇન પર રોકી રાખે છે [MDN].

4. Network Request Orchestration: Live fetch() API

હવે આપણે જે સ્વિગી (Swiggy) જેવો લાઇવ ડેટા લાવવાનો કોડ શરૂઆતમાં જોયો હતો, તેને મોર્ડન પ્રોફેશનલ સ્ટાઇલમાં try...catch એરર હેન્ડલિંગ સાથે લખતા શીખીએ [JavaScript.info -> Network requests]:

```
javascript
```

```
// async ફંક્શન બનાવ્યું
```

```
async function getFoodData() {
```

```
try {  
    const swiggyUrl = "https://swiggy.com";  
  
    // ૧. fetch() દ્વારા ડેટા મંગાવ્યો (અહીં await પ્રોમિસ પૂરું થવાની રાહ જોશે)  
    const response = await fetch("https://swiggy.com");  
  
    // ૨. ચેક કરો કે નેટવર્ક રિક્વેસ્ટ સક્સેસફુલ છે કે નહીં [JavaScript.info -> Network requests]  
    if (!response.ok) {  
        throw new Error(`સર્વર કનેક્શન ફેલ! Status: ${response.status}`);  
    }  
  
    // ૩. રિસ્પોન્સને JSON ઓબ્જેક્ટમાં કન્વર્ટ કર્યો  
    const data = await response.json();  
  
    // ૪. અસલી ડેટા એક્સ્ટ્રેક્ટ કર્યો  
    let whatsOnYourMind = data.data.cards[0].card.card.imageGridCards.info;  
    console.log("સફળતાપૂર્વક ડેટા મળી ગયો:", whatsOnYourMind);  
  
    // અહીંથી તમે તમારી DOM Manipulation (સેક્શન ૨ મુજબ) વાપરીને કાર્ડ્સ રેન્ડર કરી શકો છો!  
  
    } catch (error) {  
        // જો ઇન્ટરનેટ બંધ હોય કે API બગડી ગઈ હોય, તો આખો કોડ કેશ થવાના બદલે અહીં સેફ હેન્ડલ થશે  
        [MDN]  
        console.error("એરર પકડાઈ ગઈ:", error.message);  
    }  
}  
  
// ફંક્શન કોલ કરવું  
getFoodData();  
  
Use code with caution.
```

તમે હવે અસિંક્રોનસ JavaScript ના સૌથી પાવરફુલ ટૂલ્સ—Promises, Async/Await, અને લાઈવ API Handling ને એકદમ પ્રોફેશનલ સ્ટાન્ડર્ડ સાથે સમજી ગયા છો.

હવે આપણે આ અલ્ટીમેટ રોડમેપના સૌથી છેલ્લા માઇલસ્ટોન **Section 3: Topic 4 & 5 (Performance Optimization જેમ કે Debouncing & Throttling, Web Storage, Memory Leaks અને Security)** તરફ આગળ વધીએ?

જ્યારે પણ તમે આ ફાઇનલ વિજય માટે તૈયાર હોવ, "Next" અથવા "હા" લખો!

ખૂબ જ સરસ! હવે આપણે આવી ગયા છીએ આપણા અલ્ટીમેટ રોડમેપના સૌથી છેલ્લા અને માસ્ટર સેક્શન તરફ—**Section 3: Topic 4 & 5 — Performance, Security & Expert Techniques**.

આ એડવાન્સ ટોપિક્સ તમને એક સામાન્ય કોડરથી અલગ પાડીને એક **પ્રોફેશનલ આર્કિટેક્ટ (Ninja)** બનાવશે, જે માત્ર કોડ લખતો નથી પણ તે એપ્લિકેશનને સુપર-ફાસ્ટ અને સિક્યોર પણ બનાવે છે.

● Section 3: Topic 4 & 5 — Performance, Security & Expert Techniques

1. Performance Optimization: Debouncing & Throttling

જ્યારે વેબપેજ પર કોઈ ઇવેન્ટ (જેમ કે ટાઇપિંગ કે સ્ક્રોલ) સેકન્ડમાં ૧૦૦ વાર ટ્રિગર થતી હોય, ત્યારે જો આપણે દર વખતે ફંક્શન રન કરીએ તો બ્રાઉઝર હેંગ થઈ જાય. આને ઓપ્ટિમાઇઝ કરવા માટે બે ટેકનિક વપરાય છે [GeeksforGeeks]:

A. Debouncing (ડીબાઉન્સિંગ)

નિયમ: "યુઝર જ્યાં સુધી ટાઇપિંગ કરવાનું કે એક્શન કરવાનું થોડીવાર માટે બંધ ન કરે, ત્યાં સુધી ફંક્શન કોલ ન કરવું."

- **ઉદાહરણ:** Google Search Autosuggest. તમે J-A-V-A ટાઇપ કરતા હોવ ત્યારે દર અક્ષરે સર્વર પર રિક્વેસ્ટ નથી જતી, પણ તમે ટાઇપ કરવાનું બંધ કરો એટલે જ રિક્વેસ્ટ જાય છે [GeeksforGeeks].

javascript

```
function debounce(func, delay) {  
  let timerId;  
  return function (...args) {  
    clearTimeout(timerId); // અગાઉનો ટાઇમર કેન્સલ કરો  
    timerId = setTimeout(() => {  
      func.apply(this, args);  
    }, delay);  
  };  
}
```

// ઉપયોગ: સર્ચ બારમાં કી-પ્રેસ ઇવેન્ટ ઓપ્ટિમાઇઝ કરવા

```
const processSearch = debounce(() => console.log("API રિક્વેસ્ટ ગઈ!"), 500);
```

Use code with caution.

B. Throttling (થ્રોટલિંગ)

નિયમ: "ભલે યુઝર ગમે એટલી સ્પીડમાં એક્શન કરે, આપણે ફંક્શનને ચોક્કસ સમયગાળામાં (જેમ કે ૬૨૫૦૦ મિલિસેકન્ડ) વધુમાં વધુ એક જ વાર રન થવા દઈશું."

- ઉદાહરણ:** ગેમિંગમાં બંદૂક ચલાવવાનું બટન કે વેબપેજનું લાઈવ સ્ક્રોલિંગ (Window Scroll) [GeeksforGeeks].

2. Web Storage Ecosystem (ડેટા સાચવવાની સિસ્ટમ)

બ્રાઉઝરમાં ડેટા સાચવવા માટે મુખ્યત્વે ૩ ઓપ્શન્સ છે [W3Schools]:

સ્ટોરેજ પ્રકાર	સ્ટોરેજ કેપેસિટી	ક્યાં સુધી જીવશે? (Expiry)	સિક્યોરિટી રિસ્ક?	બેસ્ટ યુઝ કેસ
localStorage	~5MB - 10MB	ક્યારેય ડિલીટ નહીં થાય (જ્યાં સુધી કોડથી ન કરો) [W3Schools].	XSS (કોઈપણ JS કોડ આ વાંચી શકે).	યુઝરની થીમ પ્રિફરન્સ (Dark/Light Mode).
sessionStorage	~5MB	ટેબ (Tab) કે બ્રાઉઝર બંધ થતાં જ ગાયબ થાય [W3Schools].	XSS રિસ્ક.	એક જ સેશનની ટેમ્પરરી માહિતી (જેમ કે ફોર્મ ડેટા).
Cookies	~4KB	એક્સપાયરી ડેટ સેટ કરી શકાય [W3Schools].	HttpOnly સેટ કરવાથી JS આને ચોરી શકતું નથી.	યુઝરના લોગીન ટોકન્સ (JWT/Auth tokens) સાચવવા [W3Schools].

LocalStorage કોડ એક્ઝામ્પલ:

```
javascript
```

```
localStorage.setItem("theme", "dark"); // સેવ કરવું
```

```
const currentTheme = localStorage.getItem("theme"); // રીડ કરવું
```

```
localStorage.removeItem("theme"); // ડિલીટ કરવું
```

Use code with caution.

3. Modern Modularity (ESM - કોડના ટુકડા કરવા)

મોટા પ્રોજેક્ટ્સમાં આખો કોડ એક જ ફાઇલમાં લખવાના બદલે અલગ-અલગ ફાઇલોમાં વહેંચવામાં આવે છે [MDN]:

```
javascript
```

```
// ફાઇલ ૧: mathUtils.js

export const add = (a, b) => a + b; // Named Export

export default function multiply(a, b) { return a * b; } // Default Export
```

```
// ફાઇલ ૨: main.js

import multiply, { add } from './mathUtils.js'; // કોડ ઇમ્પોર્ટ કરવો [MDN]

console.log(add(5, 5));

Use code with caution.
```

4. Memory Optimization & Leaks (મેમરીની ચોરી રોકવી)

બ્રાઉઝરનું એન્જિન જે ડેટા વપરાશમાં ન હોય તેને **Garbage Collection (Mark-and-Sweep Algorithm)** દ્વારા ઓટોમેટિક ક્લીન કરે છે [MDN -> Memory Management]. પરંતુ ઘણીવાર ડેવલપરની ભૂલોના કારણે મેમરી લીક (Memory Leaks) થાય છે અને વેબપેજ ધીમું પડી જાય છે:

- **Uncleared Timers:** જો તમે setInterval ચાલુ કરીને તેને clearInterval કરવાનું ભૂલી જાવ, તો તે બેકગ્રાઉન્ડમાં ચાલુ જ રહેશે [MDN -> Memory Management].
- **Forgotten Event Listeners:** કોઈ HTML એલિમેન્ટને પેજમાંથી રીમુવ (remove()) કરી દીધા પછી પણ જો તેની સાથે જોડેલો addEventListener હટાવ્યો ન હોય, તો તે મેમરી રોકી રાખશે.

5. Modern JS Engines Advanced Objects: Proxy API

એડવાન્સ ડેવલપર્સ અને ફ્રેમવર્ક્સ (જેમ કે Vue 3) ઓબ્જેક્ટના બિહેવિયરને કસ્ટમાઇઝ કરવા માટે Proxy નો ઉપયોગ કરે છે [JS.info]. આ એક એવો મિડલમેન (ચોકીદાર) ઓબ્જેક્ટ છે જે અસલી ઓબ્જેક્ટ પર થતી કોઈપણ એક્શનને ઇન્ટરસેપ્ટ (રોકી) શકે છે:

```
javascript

const user = { name: "Raj", age: 22 };

const userProxy = new Proxy(user, {

  get(target, property) {

    console.log(`પ્રોપર્ટી '${property}' રીડ થઈ રહી છે...`);

    return target[property];

  },

  set(target, property, value) {

    if (property === "age" && value < 0) {
```

```
        throw new Error("ઉંમર ક્યારેય માઇનસમાં ન હોઈ શકે!"); // વેલિડેશન સુરક્ષા!  
    }  
    target[property] = value;  
    return true;  
}  
});
```

```
console.log(userProxy.name); // રીડ વખતે લોગ દેખાશે  
userProxy.age = -5; // સીધો ક્રેશ થશે અને ખોટો ડેટા સ્ટોર થતો રોકશે!  
Use code with caution.
```

🎉 અભિનંદન! તમે JavaScript રીડમેપ સફળતાપૂર્વક પૂર્ણ કર્યો છે! 🎉

તમે એક શૂન્ય લેવલ (**Novel**) થી શરૂ કરીને એન્જિન કોર, DOM, ઇવેન્ટ્સ, અસિંક્રોનસ JS, OOP, અને પર્ફોર્મન્સ ઓપ્ટિમાઇઝેશન સુધીના તમામ **Ninja** લેવલના કોન્સેપ્ટ્સ ઓફિશિયલ વેબ ડોક્યુમેન્ટેશનના સ્ટાન્ડર્ડ સાથે કમ્પ્લીટ કરી લીધા છે.

હવે તમે કોઈપણ મોડર્ન ફ્રન્ટએન્ડ ફ્રેમવર્ક જેમ કે **React.js**, **Vue.js** અથવા બેકએન્ડ માટે **Node.js** શીખવા માટે ૧૦૦% ઓફિશિયલી લાયક બની ગયા છો!